

Architecture & maintenabilité d'ACRA

Analyse de l'architecture logicielle, de la qualité du code, de la testabilité et de la dette technique de l'application, en vue de la publication v1.0.

Stack Next.js 16 (App Router) · React 18 · TypeScript · Prisma · PostgreSQL · Tailwind

Volumétrie ~31 700 lignes · 128 fichiers source · 20 modèles · 27 routes API · 21 pages

Qualité 23 fichiers de test · 229 tests Vitest · CI GitHub Actions · Docker

Date 14 juin 2026 · **Version** 1.0 · [Pré-publication](#)

1. Synthèse exécutive

ACRA repose sur une **architecture moderne, cohérente et bien testée**. La séparation des responsabilités (logique métier isolée en `lib/`, validation par schémas, patterns réutilisables) favorise la maintenabilité, et la couverture de tests sur la logique critique est solide. La principale dette identifiée est la **taille de quelques fichiers** (i18n, page de configuration, ateliers) et un contournement technique pour l'export PDF. Rien de bloquant pour une v1 ; un plan de refactorisation léger est proposé.

B+

NOTE MAINTENABILITÉ

229

TESTS VERTS

20

MODÈLES DE DONNÉES

4/4

CHECKS CI

2. Vue d'ensemble de l'architecture

Couche	Rôle & implémentation
Présentation	Next.js App Router — 21 pages, 26 composants React (dont composants d'ateliers), Tailwind + design system, thème clair/sombre, i18n typé
API / contrôleurs	27 <i>route handlers</i> Next ; auth via next-auth ; validation zod ; garde RBAC <code>requireAdmin</code>
Logique métier	Isolée dans <code>src/lib/</code> : échelles/matrice, permissions, exemples (schema-driven), MFA, crypto, rate-limit, i18n
Données	Prisma ORM, PostgreSQL, 20 modèles, 26 migrations versionnées ; singletons de configuration (<code>id="global"</code>)
Transverse	Journalisation d'audit, chiffrement des secrets, middleware d'auth, en-têtes de sécurité

3. Patterns & bonnes pratiques observés

- **Logique pure isolée et testable** — fonctions sans I/O dans `lib/` (échelles, OTP, permissions, contraste), testées sans base de données.
- **Configuration *schema-driven*** — registre de catégories d'exemples décrivant champs et validation, exploité par l'UI générique et le *sanitizer* serveur (extensible sans code spécifique).
- **Pattern singleton de configuration** — modèles `Configuration`, `OrganizationConfig`, `SSOConfig`, `SMTPConfig` (`id="global"`) + API GET/PUT homogènes.
- **Sécurité par conception** — secrets chiffrés (`secret-crypto`), audit redacté, validation centralisée.
- **Internationalisation typée** — type dérivé du fichier FR garantissant la complétude des 5 langues à la compilation.
- **TDD** — tests écrits avant l'implémentation (consigne projet), 229 tests verts.

4. Qualité & outillage

Dimension	État	Détail
Typage	Fort	TypeScript strict, <code>tsc --noEmit</code> propre, bloquant en CI
Tests	Solide	229 tests (permissions, crypto, échelles, anti-injection CSV, cycle de vie comptes, MFA...)
Intégration continue	En place	4 jobs : tests, typecheck, npm audit (prod), scan secrets/.env
Conteneurisation	Oui	Docker + docker-compose (app, db, migrator), build prod sain
Migrations	Versionnées	26 migrations Prisma, <code>migrate deploy</code> au démarrage

5. Dette technique & points de vigilance

Réf.	Sévérité	Constat	Recommandation
A-1	Moyen	Fichiers volumineux : <code>configuration/page.tsx</code> (~1320 l.), <code>Atelier1/5.tsx</code> (~900-950 l.), <code>i18n/*.ts</code> (~1446 l. x5)	Découper en sous-composants/hooks ; scinder les bundles i18n par domaine
A-2	Moyen	Export PDF : précompilation esbuild + <code>createRequire</code> au runtime (contournement de l'incompatibilité du transform JSX SWC avec <code>@react-pdf</code>)	Documenter/encapsuler ; surveiller les MAJ Next/ <code>@react-pdf</code> pour retirer le contournement
A-3	Faible	État rate-limit en mémoire (non distribué)	Store externe (Redis) pour le multi-instance
A-4	Faible	Accès Prisma via <code>(prisma as any)</code> sur quelques nouveaux modèles	Régénérer/typer après stabilisation du client
A-5	Info	3 CVE high dans la chaîne de build (esbuild/vite/vitest, devDeps)	Non expédié ; planifier la montée <code>@vitejs/plugin-react</code> (majeur)

6. Métriques de maintenabilité

Indicateur	Valeur	Appréciation
Lignes de code (hors tests)	~31 700	Cohérent pour le périmètre fonctionnel
Ratio fichiers testés / logique critique	23 suites	Bon — couvre la logique métier sensible
Couplage UI ↔ métier	Faible	Bon — métier en <code>lib/</code>
Fichiers > 800 lignes	~10	À surveiller
Dépendances de production	14	Maîtrisé

7. Plan de refactorisation recommandé (post-v1)

1. **Découpage de `configuration/page.tsx`** en sous-composants par section (échelles, options, exemples) + hooks de chargement/sauvegarde.
2. **Scission des fichiers `i18n`** par domaine fonctionnel (auth, ateliers, admin, config) pour réduire la taille unitaire.
3. **Extraction des gros composants d'atelier** (sous-sections, formulaires) en unités réutilisables et testables.
4. **Encapsulation du pipeline PDF** et veille sur le retrait du contournement esbuild.
5. **Externalisation du rate-limit** (Redis) en vue d'un déploiement multi-instance.

8. Conclusion

Verdict : architecture saine, publiable en v1. Les choix techniques sont modernes et cohérents, la logique métier est isolée et testée, la CI verrouille la qualité. La dette technique est limitée, identifiée et adressable de façon incrémentale — principalement la taille de quelques fichiers et un contournement PDF à encapsuler. Aucun obstacle structurel à la mise en production.

Analyse statique et revue d'architecture réalisées le 14/06/2026 sur la branche `main` (Next.js 16, Prisma 5, 229 tests).